

MÁSTER EN ROBÓTICA Y AUTOMATIZACIÓN
LABORATORIO DE ROBÓTICA

Preparación práctica laboratorio UR3 – Moveit & Gazebo

Realizado por:

Glen Alejandro Guerrero Burbano

Profesor:

Daniel Vicente Rodrigo Muñoz

05 de junio de 2026

I. RESUMEN

En este trabajo se presenta la preparación de un entorno de simulación para un robot Universal Robots UR3 equipado con un gripper paralelo de dos dedos. El sistema integra ROS 2 Jazzy, MoveIt 2 y Gazebo Harmonic para realizar tareas de pick-and-place dentro de una celda robótica. La arquitectura se organiza en tres paquetes independientes: descripción de la celda con el modelo del UR3 (`robot_description_ur3`), configuración de MoveIt generada con Setup Assistant (`robot_cell_moveit_ur3`) y un paquete propio de control y simulación física (`robot_control_ur3`) que arranca todo el sistema mediante un único launch. Se importan dos objetos manipulables, una caja y un basurero, y se definen poses guardadas en el SRDF que conforman el flujo completo de agarre y transporte. El resultado es un sistema funcional capaz de planificar trayectorias en RViz y ejecutarlas físicamente en Gazebo, abriendo la puerta a la automatización completa del pick-and-place.

II. MATERIALES Y MÉTODOS:

El proyecto se desarrolla sobre Ubuntu 24.04 ejecutándose en una máquina virtual VirtualBox sobre Windows 11. El entorno de trabajo es un contenedor Docker que proporciona ROS 2 Jazzy Jalisco, MoveIt 2, Gazebo Harmonic y el plugin de integración `gz_ros2_control`. Todo el código fuente reside en un workspace de colcon estructurado en tres paquetes ROS 2 dentro de la carpeta `robot_cell` del directorio `src`, lo que permite mantener separadas las capas de descripción, planificación y control.

La estructura de paquetes es la siguiente: `robot_description_ur3` contiene el URDF/Xacro del UR3 con la celda completa (mesa, pedestal, paredes, robot mount y gripper), las mallas en formato STL y DAE, los modelos SDF de los objetos manipulables y el archivo del mundo de Gazebo; `robot_cell_moveit_ur3` es la configuración de MoveIt generada con Setup Assistant a partir del URDF del UR3 y contiene el SRDF, los archivos de kinematics, `joint_limits`, `moveit_controllers` y los launches estándar de MoveIt; y `robot_control_ur3` es el paquete propio que gestiona la simulación física, conteniendo el URDF de control con el plugin `gz_ros2_control`, el YAML de controladores y el launch principal que arranca toda la pila.

PARTE 1 – DESCRIPCIÓN CELDA - `robot_description_ur3`: El paquete de descripción se construye a partir del macro oficial de Universal Robots, parametrizando el tipo del robot como `ur3`. Se incluyen todos los parámetros físicos del UR3 cargados desde `ur_description` (`joint_limits`, `default_kinematics`, `physical_parameters` y `visual_parameters`). Alrededor del robot se instancia el resto de la celda mediante el macro `robot_cell`, que añade el pedestal sobre el que se monta el UR3, el suelo, las paredes laterales y posteriores, un pallet decorativo y el adaptador `robot_mount`. El gripper se monta al final del eslabón `tool0` del UR3, configurado como una pinza paralela de dos dedos con un grado de libertad activo (`g_left_joint`) y el dedo derecho como joint mimic del izquierdo, garantizando un movimiento simétrico.

Los objetos manipulables se han modelado como modelos SDF independientes en la subcarpeta models del paquete, lo que permite spawnearlos como cuerpos dinámicos separados del robot. Se han preparado dos objetos: una caja de 50×50×50 mm con masa de 150 g y colisión geométrica de tipo box; y un basurero que actúa como destino del pick-and-place, definido como modelo estático con colisión cilíndrica y malla visual. Esta separación entre objetos manipulables y la cadena cinemática del robot es fundamental porque permite que Gazebo simule la física de los objetos de forma realista, mientras MoveIt continúa planificando únicamente con la geometría del robot.

El CMakeLists.txt del paquete instala todos los directorios de recursos en share, de modo que tanto MoveIt como Gazebo puedan resolver las rutas package:// que aparecen en los URDF y SDF:

Directorio	Contenido
launch	Lanzadores auxiliares (visualización en RViz, etc.)
meshes	Mallas STL/DAE del robot y los objetos
models	Modelos SDF independientes (caja, basurero)
urdf	URDF/Xacro del robot y la celda
worlds	Mundo de Gazebo cell_world.sdf
rviz	Configuraciones de RViz

Tabla 1. Contenido del paquete robot_description_ur3.

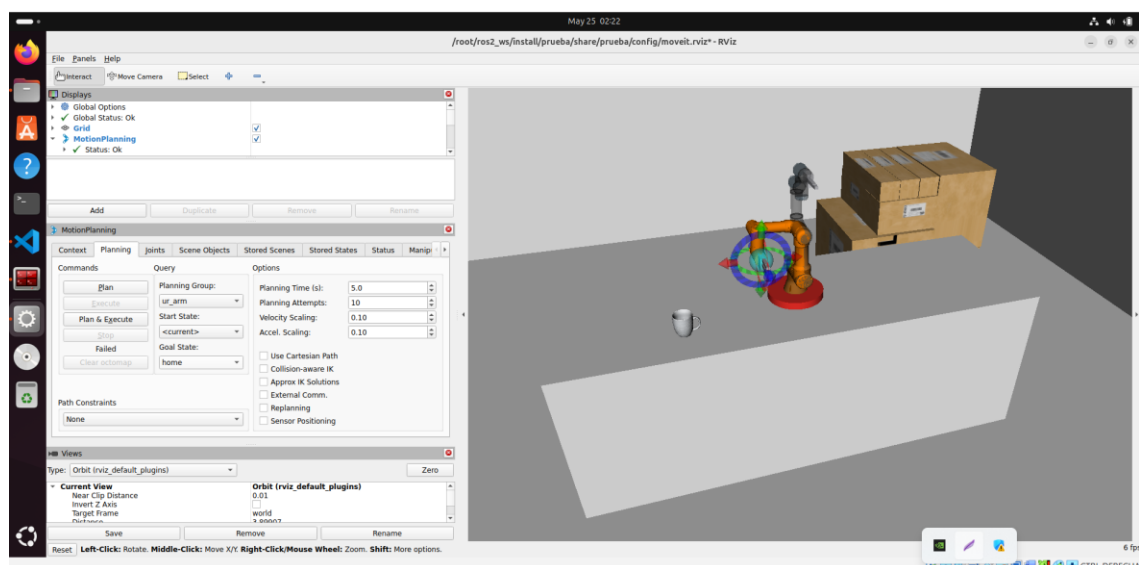


Figura 1. RViz UR3

PARTE 2 – CONFIG MOVEIT - robot_cell_moveit_ur4: La configuración de MoveIt se genera mediante MoveIt Setup Assistant cargando el URDF del UR3 con el gripper. El asistente automatiza la creación del SRDF (matriz de auto-colisiones, grupos de planificación, estado pasivo del joint mimic), los archivos de kinematics y joint_limits y los launches estándar de MoveIt. Se han definido dos grupos de planificación: ur_arm, que agrupa los seis joints del brazo (shoulder_pan, shoulder_lift, elbow, wrist_1, wrist_2, wrist_3 con prefijo ur3_) y utiliza KDLKinematicsPlugin como solver de cinemática inversa; y gripper, que contiene únicamente el g_left_joint, siendo el g_right_joint passive como mimic del primero.

Sobre la configuración generada por el asistente se han realizado dos ajustes finos necesarios para que la planificación funcione de forma consistente. En kinematics.yaml se ha aumentado el kinematics_solver_timeout a 0.05 segundos y añadido kinematics_solver_attempts: 3, lo que mejora notablemente la tasa de éxito del solver KDL. En joint_limits.yaml se han activado los límites de aceleración con has_acceleration_limits: true asignando valores de 5 rad/s² para los joints del hombro y codo y 10 rad/s² para los wrists; esta configuración es imprescindible para que el adapter AddTimeOptimalParameterization de MoveIt pueda calcular el perfil temporal de las trayectorias sin errores.

Las poses guardadas en el SRDF representan los puntos clave del flujo de pick-and-place. Se han creado capturando manualmente desde RViz los valores articulares deseados (en grados desde la pestaña Joints), convirtiéndolos a radianes y añadiéndolos como bloques <group_state> al archivo robot_cell.srdf:

Pose	Grupo	Descripción
init	ur_arm	Pose inicial neutra del robot al arrancar
home	ur_arm	Pose elevada de reposo segura entre tareas
cerca_B	ur_arm	Aproximación sobre la caja (pre-grasp y elevación post-grasp)
cerca_C	ur_arm	Pose de agarre, gripper a ambos lados de la caja
drop_off	ur_arm	Pose de soltado sobre el basurero
open	gripper	Gripper completamente abierto (g_left_joint = 0.045 m)

close	gripper	Gripper cerrado para sujetar la caja ($g_left_joint \approx 0.009$ m)
-------	---------	--

Tabla 2. Poses guardadas

El uso conjunto de `cerca_B` como pose de aproximación previa al agarre y como pose de elevación después de cerrar el gripper es intencional: garantiza que el robot suba en línea aproximadamente vertical tras agarrar la caja, evitando trayectorias laterales que tenderían a hacer resbalar el objeto.

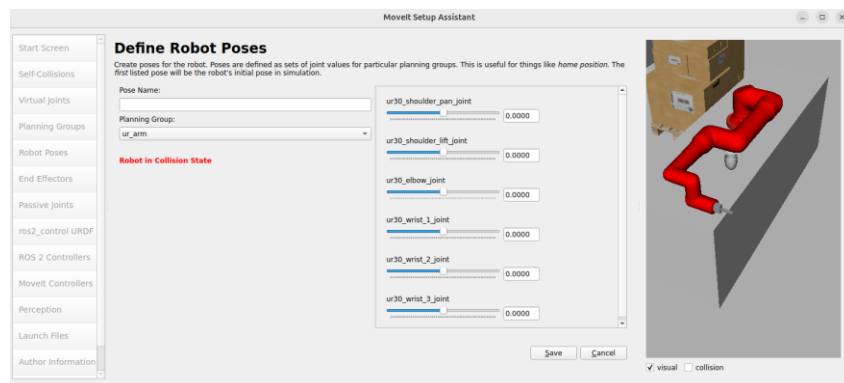


Fig 2. Config moveit

PARTE 3 – SIMULACIÓN CONTROL GZ – robot_control_ur3:

El paquete `robot_control_ur3` se ha creado basado en la carpeta suministrada por el docente para el `ur30`, para gestionar exclusivamente la simulación en Gazebo. Su pieza central es el archivo `robot_cell_control.urdf.xacro`, un URDF de simulación que reutiliza la geometría del paquete de descripción y añade dos elementos críticos: un bloque `<ros2_control>` que declara los siete joints actuados (los seis del UR3 y `g_left_joint` del gripper) usando como sistema de hardware `gz_ros2_control/GazeboSimSystem`, y un bloque `<gazebo><plugin>` que carga la librería `libgz_ros2_control-system.so`. Este plugin es quien arranca el `controller_manager` dentro de la simulación de Gazebo, lo que permite que `ros2_control` gestione físicamente los joints del robot simulado.

Cada joint del bloque `ros2_control` expone una interfaz de comando de posición y reporta su posición y velocidad. Los valores `initial_value` definen la pose con la que el robot arranca en la simulación física. Para el gripper se ha fijado `g_left_joint` con un valor inicial de 0.045 (abierto) para garantizar que ningún contacto inicial con la caja perturbe la simulación.

El archivo `controller.yaml` configura dos controladores del paquete `joint_trajectory_controller`. El primero, `ur_arm_controller`, gestiona los seis joints del UR3 con interfaz de comando de posición. El segundo, `gripper_controller`, gestiona únicamente `g_left_joint` y está configurado con un constraint generoso (`goal_time: 5.0` y tolerancia `g_left_joint goal: 0.020`) para evitar fallos cuando la pinza se cierra contra un objeto. Los nombres de estos controladores coinciden exactamente con los declarados en

el `moveit_controllers.yaml` del paquete de MoveIt, lo que garantiza que las trayectorias planificadas se enruten correctamente al hardware simulado.

El launch principal `gz_moveit_ur3.launch.py` orquesta el arranque secuencial de toda la pila utilizando `TimerAction` para evitar condiciones de carrera. La secuencia es la siguiente:

- Variables de entorno de Gazebo (`GZ_SIM_MODEL_PATH` y compañía) para que el motor encuentre los modelos y mallas referenciados con `package://`.
- Servidor y cliente de Gazebo Harmonic cargando el mundo `cell_world.sdf`.
- `robot_state_publisher` con el URDF procesado por `xacro`.
- Spawn del robot en Gazebo a partir del topic `/robot_description`.
- Spawn de la caja como modelo SDF en la posición (0.3, 0.3, 0.91).
- Spawn del basurero en (-0.4, -0.15, 1.0) con una rotación de `roll = 1.55 rad` para orientarlo correctamente.
- Bridge `ros_gz_bridge` para sincronizar el topic `/clock` entre Gazebo y ROS.
- Spawners de los tres controladores: `joint_state_broadcaster`, `ur_arm_controller` y `gripper_controller`.
- `MoveGroup` de MoveIt incluyendo el launch `move_group.launch.py` del paquete de MoveIt.
- RViz con la configuración generada por el Setup Assistant.

Con un único comando, `ros2 launch robot_control_ur3 gz_moveit_ur3.launch.py`, se arrancan los tres entornos (Gazebo, MoveIt y RViz) plenamente integrados y listos para planificar y ejecutar trayectorias.

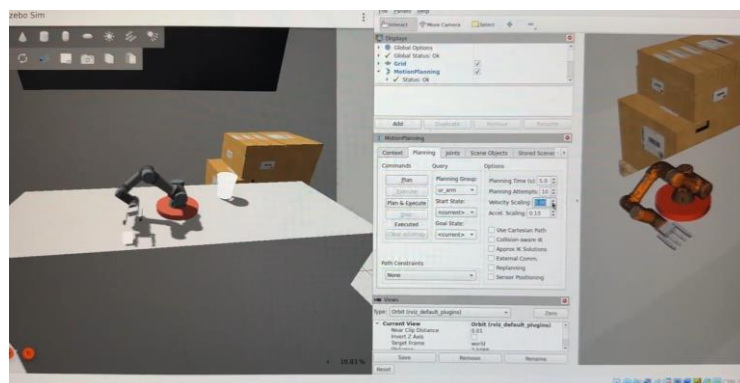


Figura 3. GZ

PARTE 4 – Flujo PICK AND PLACE:

Una vez el sistema está lanzado, la operación completa de pick-and-place se ejecuta desde RViz seleccionando la pose objetivo en el panel MotionPlanning y pulsando Plan & Execute. El flujo lógico es: ir a home como pose segura inicial, acercarse a la caja con `cerca_B`, abrir el gripper con la pose `open`, descender a la pose de agarre `cerca_C`, cerrar el gripper con la pose `close` para sujetar la caja, elevar de nuevo a `cerca_B` (esta vez con

la caja agarrada), trasladar al destino con drop_off, abrir el gripper con open para soltar la caja sobre el basurero y volver a home.

Durante las fases de transporte con la caja agarrada, se recomienda reducir los factores Velocity Scaling y Acceleration Scaling del panel MotionPlanning de RViz al 5% para minimizar las fuerzas inerciales que actúan sobre el objeto. El agarre por fricción se ha calibrado físicamente ajustando la masa de los dedos del gripper a 0.25 kg cada uno, el effort del joint prismático a 500 N y el coeficiente de fricción μ a 3.0 tanto en los bloques <gazebo reference> del dedo izquierdo y derecho como en la superficie de colisión de la caja. Esta combinación de masa, fuerza y fricción permite que el agarre se mantenga durante el transporte sin necesidad de utilizar un plugin de unión rígida adicional.

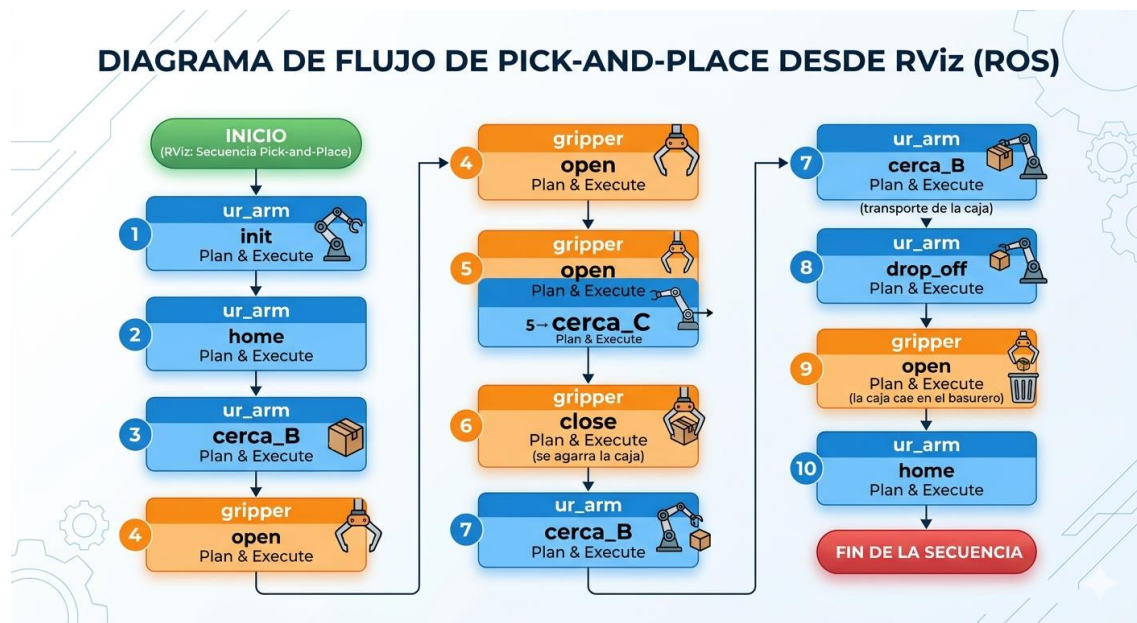


Figura 4. Secuencia usada en el video

Enlace repositorio gitHub: Se comparte el enlace del repositorio en github el cual es público y se puede ver los paquetes, readme con descripción también detallada como en este informe y el enlace del video sustentando el código y la simulación.

Repositorio original git hub: <https://github.com/glenguerrero01/Celda-UR-con-RViz-Gazebo-y-Moveit#ejecutar-la-secuencia-de-pick-and-place-desde-rviz>

Video explicación: <https://youtu.be/Y9if-JQWyHw>

III. CONCLUSIONES

La división del sistema en tres paquetes ROS 2 independientes ha resultado clave para mantener un proyecto manejable y escalable. Separar la descripción geométrica, la planificación con MoveIt y el control en simulación permite trabajar y depurar cada capa por separado, e incluso intercambiar componentes (por ejemplo, sustituir el robot UR3 por otro modelo de la familia UR) modificando un único parámetro xacro sin afectar al resto del sistema.

El uso de MoveIt Setup Assistant para generar la configuración inicial ha demostrado ser muy productivo, ya que automatiza la creación de la matriz de auto-colisiones, el SRDF y los launches estándar. Sin embargo, la configuración por defecto requiere ajustes finos para funcionar correctamente: la habilitación de límites de aceleración en `joint_limits.yaml` es imprescindible para que el parametrizador temporal de MoveIt funcione, y el aumento del timeout y del número de intentos del solver KDL en `kinematics.yaml` mejora notablemente la consistencia de la planificación. Estos detalles, aunque pequeños, marcan la diferencia entre un sistema que planifica de forma robusta y uno que falla con frecuencia.

La integración entre MoveIt y Gazebo a través de `gz_ros2_control` y el `JointTrajectoryController` es directa una vez se respeta la convención de nomenclatura entre el `moveit_controllers.yaml` y el `controller.yaml` del paquete de control. Mantener nombres de controladores coherentes en ambos paquetes (`ur_arm_controller` y `gripper_controller` en este proyecto) evita errores difíciles de diagnosticar como la imposibilidad de enviar trayectorias al hardware simulado.

El uso de modelos SDF independientes para los objetos manipulables (caja y basurero), en lugar de definirlos como links fijos del URDF del robot, ha sido una decisión arquitectónica acertada. Permite que Gazebo trate estos objetos como cuerpos dinámicos con masa, inercia y fricción reales, lo que abre la posibilidad de que respondan físicamente al contacto con el robot. La contrapartida es que MoveIt deja de conocerlos automáticamente, por lo que para tareas más sofisticadas habría que añadirlos manualmente a la `PlanningScene` como `CollisionObject`.

Finalmente, el agarre por fricción pura, aunque más sencillo de implementar que un plugin de unión rígida, exige un compromiso fino entre masa de los dedos, fuerza del actuador, coeficiente de fricción y velocidad de transporte. Para escenarios más exigentes o cargas más pesadas, sería recomendable evaluar la incorporación del plugin `gz-sim-detachable-joint-system`, que crearía una unión rígida temporal entre el gripper y el objeto al cerrarse la pinza, garantizando un agarre perfecto independientemente de la dinámica del transporte.

IV. REFERENCIAS

Coleman, D., Sukan, I. A., Chitta, S., & Correll, N. (2014). Reducing the barrier to entry of complex robotic software: a MoveIt! case study. *Journal of Software Engineering for Robotics*, 5(1), 3-16.

gz_ros2_control. (2024). ros-controls/gz_ros2_control. https://github.com/ros-controls/gz_ros2_control

Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66), eabm6074.

MoveIt Project. (2024). MoveIt 2 Documentation. <https://moveit.picknik.ai/main/index.html>

Open Robotics. (2024). ROS 2 Jazzy Jalisco Documentation. <https://docs.ros.org/en/jazzy/>

Open Robotics. (2024). Gazebo Harmonic Documentation. <https://gazebo-sim.org/docs/harmonic/>

Universal Robots. (2024). ur_description ROS 2 package. https://github.com/UniversalRobots/Universal_Robots_ROS2_Description